



Mastering Observability

ADVISORY · STRATEGY · PRACTICE

FRAMEWORK EBOOK · METRICS & MAYHEM

The Outcome Question: can your team explain why?

When the number your CFO watches moves, can your team explain why within five minutes? One question, a twenty-minute test to find your own gap, and what to do on Monday.

Allan Mann · Author, Metrics & Mayhem · [A working companion to the book](#)

FREE FRAMEWORK EBOOK · SOURCES ARE CHAPTER 2 OF THE BOOK, A PUBLIC POST-MORTEM, A PUBLIC ESSAY AND THE AUTHOR'S OWN EXPERIENCE · WORKED EXAMPLES ARE ILLUSTRATIONS AND SAY SO

AT A GLANCE

When the number your CFO watches moves, can your team explain why within five minutes?

01 Logs, metrics and traces only describe inputs

None of them tells you whether the customer finished what they came to do.

02 Pair one outcome with the signals that move it

Then put a person's name, not a team's, on the join between them.

03 Estate-wide baselines miss the quiet services

In one fifty-service estate, a failing service went unnoticed for more than two weeks.

04 Your status view can fail with the system

Early in the 2017 S3 outage, AWS could not update its own health dashboard.

05 Twenty minutes will find your gap

The five-minute test needs one outcome and a pen, not a new tool.

01

The gap

02

The practice

03

The evidence

CLOSE

On Monday

HOW THIS WAS SOURCED

Chapter 2 of Metrics & Mayhem, AWS's public post-mortem of the 2017 S3 outage, a public essay by Charity Majors, and the author's own experience, with details changed so no organisation can be identified. No vendor surveys. The worked examples are illustrations, and are labelled as such.

Both sides of the review were right

A quarterly business review. Engineering presents first. Uptime for the quarter: 99.97 per cent. Green across the board, every service level met, and the VP of Engineering is quietly pleased.

99.97%

UPTIME, IN THE SAME QUARTER CHECKOUT-FAILURE TICKETS ROSE 12 PER CENT

Source: illustrative scene, not a real organisation

Then Customer Success stands up. Checkout-failure tickets are up 12 per cent on the quarter. Satisfaction scores for the purchase flow are down. Three enterprise accounts raised it at their last contract review.

Both presentations are accurate. Neither can explain the other's number.

FROM THE FIELD

I have watched every dashboard stay green while the people using a service could not get through it. The complaints came from customers, not from our alerting. The service was low-volume, and our anomaly detection set its baseline across the whole estate, so a small service failing barely moved the needle against the big ones. We tracked it down by hand, and only then could we see where our monitoring gaps were.

That gap, between the system being up and the customer getting through, is what this short book is about. One question closes it, and you can ask it on Monday.

THE QUESTION

One question separates observability from monitoring

When the number your CFO watches moves, can your team explain why within five minutes?

Not "is the system healthy". Not "are the dashboards green". Whether a movement in something the business cares about can be traced, quickly, to the technical cause underneath it.

If the answer is yes, you have observability in the sense that matters to you. If the answer is no, or "it depends who is on call", you have monitoring, and a gap where your explanation should be.

INPUTS AND OUTCOMES

Logs, metrics and traces only describe inputs

If you have been briefed on observability, you have heard about the three pillars: logs, metrics and traces. Logs tell you what happened, metrics tell you how much, and traces tell you where a request spent its time.

Each is useful. As a way of deciding what your organisation should be able to see, they are incomplete in a way that costs money. They describe the inputs. They say nothing about the outcome.

The most useful definition I know came from a practitioner, not a vendor. Charity Majors defines observability as the power to ask new questions of your system without shipping new code or gathering new data to ask them.¹ Monitoring watches for the failures you predicted. Observability lets you investigate the one you did not.

For a leader, the practical version is shorter. Monitoring answers "is the system healthy?" Observability answers "can the customer complete the transaction, and if not, why not?"

WHERE THE EXPLANATION LIVES

Uptime and buying live in different systems

Your engineering team measures uptime. Your customers measure whether they could buy the thing.

In most organisations those two measures live in different systems, owned by different teams, reviewed in different meetings, and connected by nothing except the hope that if one is healthy the other will be too. The space between them is where the explanation lives, and most organisations have no systematic way to cross it.

PART TWO · PAIRING

Pair each outcome with the signals that move it

Closing the gap means pairing each business outcome with the technical signals that can move it, explicitly, as an operational practice with a named owner. Not as an aspiration on a strategy slide.

Take payment completion rate: the share of customers who start a checkout and finish it. Your CFO watches it. Your board watches it. The technical signals that can move it include checkout load time at the 99th percentile, payment gateway latency, error rates from a third-party processor, authentication availability and cart session timeouts.

Each of those can drop the completion rate. Unless they are linked to it, the person who gets the call starts guessing: open a dashboard, check the logs, run a query, move between three or four tools owned by three or four teams. Fifteen minutes into an incident at 11,000 transactions a second is not a debugging exercise. It is a business event.²

Paired properly, a drop in payment completion brings its likely causes with it, ordered and in context, and the investigation starts in minutes rather than after half an hour of guessing.

That pairing crosses team, tool and budget lines that no single engineering manager controls. Which is why it is your decision to make.

ON THE DAY

On a good day, the drop arrives with its causes

It is worth being specific about the target, because "better observability" is not one.

The completion rate starts to fall. Before anyone has opened a dashboard, the drop has arrived with its likely causes attached: the signals that moved in the same window, ordered by how likely each is to explain it, with enough context for whoever gets the call to start looking in the right place.² The person responding does not need to know which team owns which tool, because the pairing already knows.

Within minutes, someone can say out loud what is happening to the customer, in words the product lead understands, and which part of the estate is responsible. The owner of the join is in the conversation from the start, not found halfway through. The finance lead hears "completion is down four points because the processor is timing out", and knows what that costs.

None of that requires the organisation to see everything. It requires one outcome to be explicable, quickly, by the people who get the call. Start there.

OUTSIDE COMMERCE

The same pairing works without a checkout

Not every organisation sells through a checkout. The pairing works the same way wherever there is an outcome someone outside engineering would recognise.

Take a business that serves partners through an API. The outcome the commercial team watches is the share of partner requests answered within the time the contract promises. Miss it for a week and it shows up in a renewal conversation, long before it shows up in an engineering review.

The technical signals that can move it are different from the checkout case, and just as specific. Response time at the 99th percentile for each partner, not averaged across all of them. Error rates by partner. Authentication and token failures. Requests rejected by rate limits. Latency from the upstream services your API calls in turn. Configuration and deployment changes in the window before the number moved.

Notice the first one. Averaged across every partner, response time can look perfectly healthy while your largest partner has a miserable week, because their traffic is a small share of the total. It is the same trap as a baseline learned across a whole estate. The pairing has to be written at the level the business feels the pain, which is usually one customer, one partner or one journey, not the average.

Chapter 2 of the book names other outcomes that pair just as well: search relevance, content delivery latency, or API response time for key partners.² The method does not change. One outcome, the signals that can move it, and a person who owns the join.

THE FIVE-MINUTE TEST

Twenty minutes with a pen will find your gap

Do this on one outcome, not ten. Twenty minutes, pen in hand. If the last answer is more than five minutes, or you had to ask someone to find out, you have found your gap.

1

OUTCOME

Write down the one business outcome your CFO would recognise.

2

SIGNALS

List every technical signal that can move it.

3

VISIBILITY

Mark each signal your team can see today. Leave the rest blank.

4

OWNER

Name who owns the connection between the outcome and those signals. A person, not a team.

5

LAST P1

Take your last P1. Write down how long it took to explain the outcome's movement, in minutes.

THE TEST, FILLED IN

An honest answer usually shows blind signals

An illustration, not a real organisation, so you can see what an honest answer looks like before you write your own.

The outcome: payment completion rate.

The signals that can move it: checkout load time at the 99th percentile, payment gateway latency, third-party processor errors, authentication availability, cart session timeouts.

What the team can see today: the first three. Authentication is monitored by another team in another tool. Nobody measures cart timeouts at all.

Who owns the connection: nobody, by name. The product lead owns the number, the platform lead owns the gateway, and the join between them belongs to whoever is on call that night.

The last P1: the completion rate fell on a Tuesday evening, and it took just over forty minutes to know why. The cause was in the authentication service, the one signal the responding team could not see.

The explanation took eight times the target

Minutes from the fall in completion to a named cause



Source: the illustration above, not a real organisation

That is a common result, and a useful one. Two blind signals, no named owner, and an explanation that took eight times longer than the target. None of it needs a new tool to fix first. It needs a name next to the join and a view of the two missing signals.

Estate-wide baselines miss the quiet services

7%

OF AROUND FIFTY MAJOR SERVICES, FAILING REGULARLY

Source: author's experience, a fifty-service estate; figures rounded

FROM THE FIELD

It was the 500s that mattered. Across around fifty major services, about 7 per cent were failing regularly. Some were used ten or twenty times a day, some once a month, but each one sat inside a longer journey, so when it failed, the journey failed. One service had been throwing 500s under the radar for more than two weeks. No events on the floor, no nothing, because the anomaly detection had never fired.

Low-volume does not mean low-value. A service used twenty times a day can still be the step that decides whether a customer finishes what they came to do. Baselines learned across a whole estate will not notice it failing, because against the busy services it never looks unusual.

Three things to check in your own estate this month:

Which customer journeys depend on a service that is used only a few times a day or a few times a month?

For each of those services, is its normal learned from its own traffic, or from the estate around it?

If one of them had been failing for a fortnight, what would have told you, other than a complaint?

If the honest answer to the last one is "nothing", you are in the same position I was, and you will find out the same way.

AMAZON S3 • 28 FEBRUARY 2017

Your status view can fail with the system

On 28 February 2017, a member of Amazon's S3 team ran an established playbook command to remove a small number of servers from an S3 subsystem in Northern Virginia. One input was entered incorrectly and a larger set of servers was removed than intended.³

What followed is the outcome question at the largest scale most of us will ever see. AWS's own post-mortem records that from the start of the event until 11:37 AM Pacific time, it could not update the individual service statuses on its own Service Health Dashboard, because the dashboard's administration console depended on S3, the service that was down. It told customers through Twitter and banner text instead. Afterwards it moved that console to run across multiple regions.³

The lesson is not about Amazon. It is that the view you rely on to tell you what is happening can depend on the thing that is failing, and you only find out on the day. Ask of every leadership view you have: what does this depend on, and would it still tell me the truth if that failed?

You cannot trust the green dashboard on its own

You cannot trust the high-level green dashboard on its own. Leadership dashboards tell you what you want to know: KPIs, OKRs, where to put your effort and your money. But when an incident happens, do you really know what is going on underneath? You have to understand the estate, and that means the right standards from the ground up, so that everything is in place. You will not be right a hundred per cent of the time. Sometimes we miss things; that is the nature of monitoring. But day to day, you should always have good monitoring standards, a good strategy and good process.

WHERE THIS BREAKS

It goes wrong in three common ways

It becomes a data-integration project. Someone builds a pipeline that feeds infrastructure metrics into a business dashboard, ships it, and declares victory. That is reporting. A view that refreshes every five minutes is a faster report, not a way to explore a failure you have never seen.

It sprawls. Trying to pair every outcome at once is expensive and exhausting, and it almost always stalls. Start with the one outcome that matters most to revenue, prove it on a real incident, then earn the right to add the next.

Nobody owns the join. Engineering owns the signals, product owns the experience, finance owns the revenue. When the pairing points at a third-party processor, someone has to decide what happens next. That is an organisational question, and it has to be answered before the incident, not during it.

NOTES

¹ Charity Majors, "Observability: A Manifesto", Honeycomb blog, March 2018, updated July 2021. See also Charity Majors, Liz Fong-Jones and George Miranda, Observability Engineering, O'Reilly Media, 2022.

² Allan Mann, Metrics & Mayhem: A CTO's Guide to Observability That Actually Works, Cleartext Press, 2026, Chapter 2, "What Observability Actually Means for a CTO".

³ Amazon Web Services, "Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region", 2017, aws.amazon.com/message/41926/.

Start with one number and one question

Before your next quarterly review, name one business outcome your stack should explain and currently does not. Then ask your team one question: when this number last moved, how long did it take us to know why?

The review at the start of this book went wrong because each side brought an answer and nobody brought a question. Next time, bring three.

- 01** **Ask engineering which business outcome their dashboards can explain, and how quickly.** If the answer is a list of uptime figures, you have your answer.
- 02** **Ask what the leadership view you are looking at depends on.** If the thing it depends on failed, would the view still tell you the truth, or would it quietly stop updating?
- 03** **Ask who owns the join between the number the board watches and the signals underneath it.** A person, by name. If the room goes quiet, that silence is the most useful thing you will hear all quarter.

Pairing needs an owner, and ownership is the next step. After that, The Signal Audit turns your last incident into a ninety-day plan.

The next framework in this series is about automation, and it depends on this one. A machine allowed to act inside a policy needs that policy written in terms of outcomes, because you cannot safely let a machine act on a signal you cannot tie to an outcome. Answer the outcome question first.

THE ONE ASK

Read Chapter 4, free.

"Who Owns What When It Breaks", from Metrics & Mayhem: the three-layer ownership model, so the join your five-minute test found gets a name next to it.

masteringobservability.com/metrics-and-mayhem/free-chapter